



Subspace-Locked, Seed-Bound Latent Authentication for Transformer Models

Ayşe Okatan^a, Berker Peköz^b

Keywords: transport layer security (TLS₁), elliptic-curve Diffie–Hellman ephemeral (ECDHE₂), message queuing telemetry transport (MQTT₃), Internet of Things (IoT₄), Transmission Control Protocol (TCP₅), Hypertext Transfer Protocol Secure (HTTPS₆), Secure Shell (SSH₇), Pretty Good Privacy (PGP₈), Virtual Private Network (VPN₉), Quick UDP Internet Connections (QUIC₁₀), Rivest–Shamir–Adleman (RSA₁₁), Internet Key Exchange (IKE₁₂), Internet Protocol Security (IPsec₁₃)

The Big Idea

Every machine learning model stores what it has learned in its **weights**, millions of numbers that define how it thinks. Those weights create a geometric structure inside the model. Our IEEE paper, *Keys in the Weights* [1], showed that this geometry can serve as a cryptographic key. This poster presents the next step: we turn that concept into a full **key management layer** for network protocols, and we test it over MQTT₃, a lightweight messaging standard used in IoT₄.

Background: How Encryption Works Today

When two devices want to talk securely, they run a **key exchange**, a handshake that creates a shared secret so they can scramble their data. Today this handshake relies on abstract math (curves, large primes) that has **nothing to do with the software on the device**. We replace that abstract math with the geometry already sitting inside the model's weights.

Today's Key Exchange Methods

Method	How it creates a key	Used by
RSA₁₁	Factor two large primes	TLS ₁ (legacy), SSH ₇ , PGP ₈
Diffie–Hellman	Shared secret from modular exponents	VPNs (IKE ₁₂ /IPsec ₁₃), older TLS ₁
ECDHE₂	Shared point on an elliptic curve	TLS ₁ 1.3, HTTPS ₆ , QUIC ₁₀
Ours	Angles from transformer weight geometry	Our Application-layer key agreement protocol over MQTT ₃

All standard methods derive keys from **abstract math**, large primes, curves, or exponents that have no connection to the application. Our method is the first to derive the key directly from the model's own weights.

Table 1. Standard key exchange methods and the protocols that use them. Our method replaces abstract math with transformer weight geometry

How It Works

A model's weights define a high-dimensional space. Each device picks a random slice through that space. The **angles** between two devices' slices are always the same from both sides:

$$\cos \theta_i = \sigma_i(\mathbf{B}_A^T \mathbf{B}_B)$$

Those angles **become the encryption key**. The model's weight geometry replaces traditional key exchange math, no elliptic curves, no large primes, just the structure already inside the model.

The Key Exchange Protocol

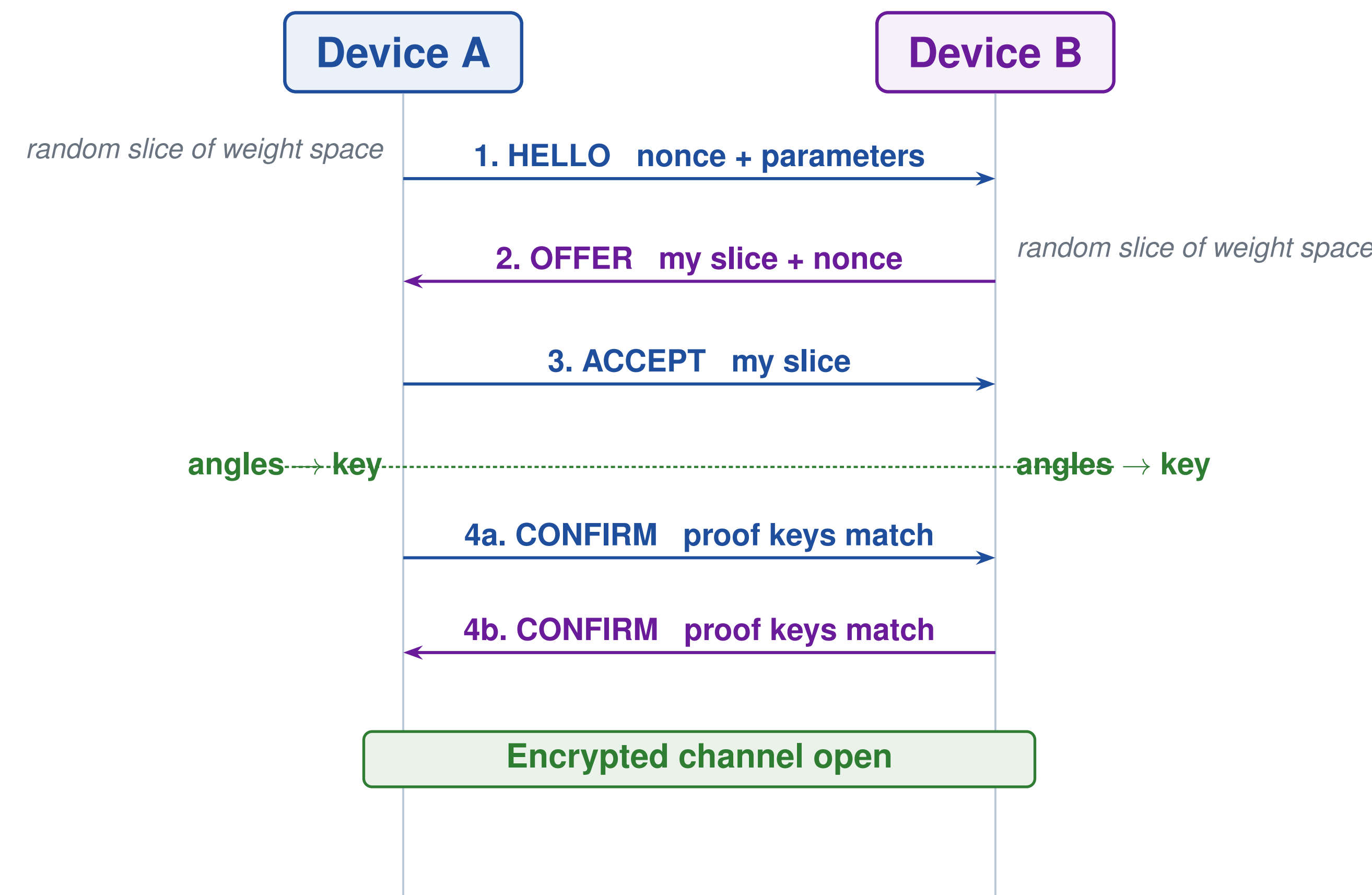


Figure 2. Four messages over MQTT₃. Both sides independently derive the same key from their model weights

Slicing the Transformer Weight Space

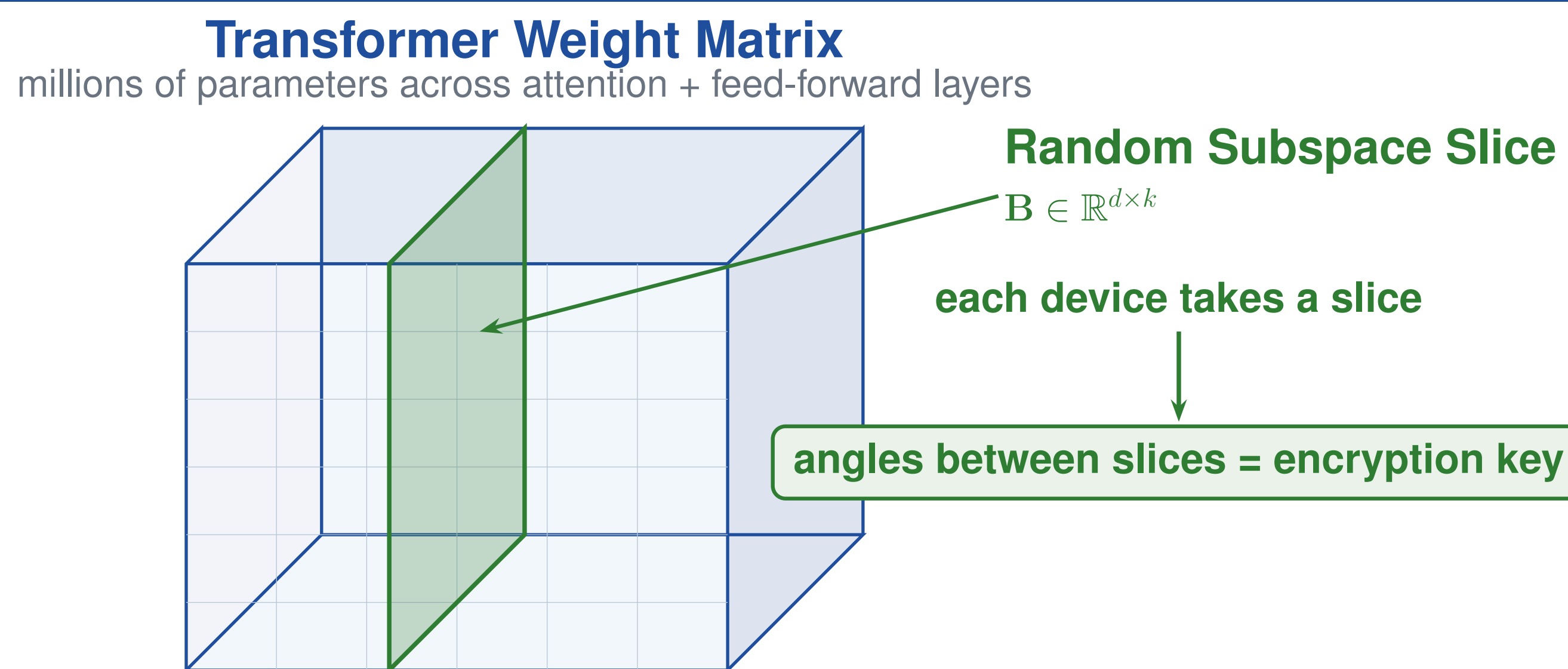


Figure 3. Each device takes a random slice through the transformer's weight matrix [1]. The angles between two devices' slices become the shared encryption key, only the real model produces the right angles

Why Transformers?

Transformer models contain stacked layers of **attention heads** and **feed-forward networks**. During training, these layers learn millions of tightly interdependent weights. The feed-forward networks alone can have hundreds of thousands of parameters per layer, and every weight is shaped by backpropagation across the entire training dataset. This creates geometry so complex that an attacker cannot reverse-engineer the slice angles without possessing the exact same trained model, **guessing the key would require recreating the entire training process**.

Standard vs. Our Protocol Architecture

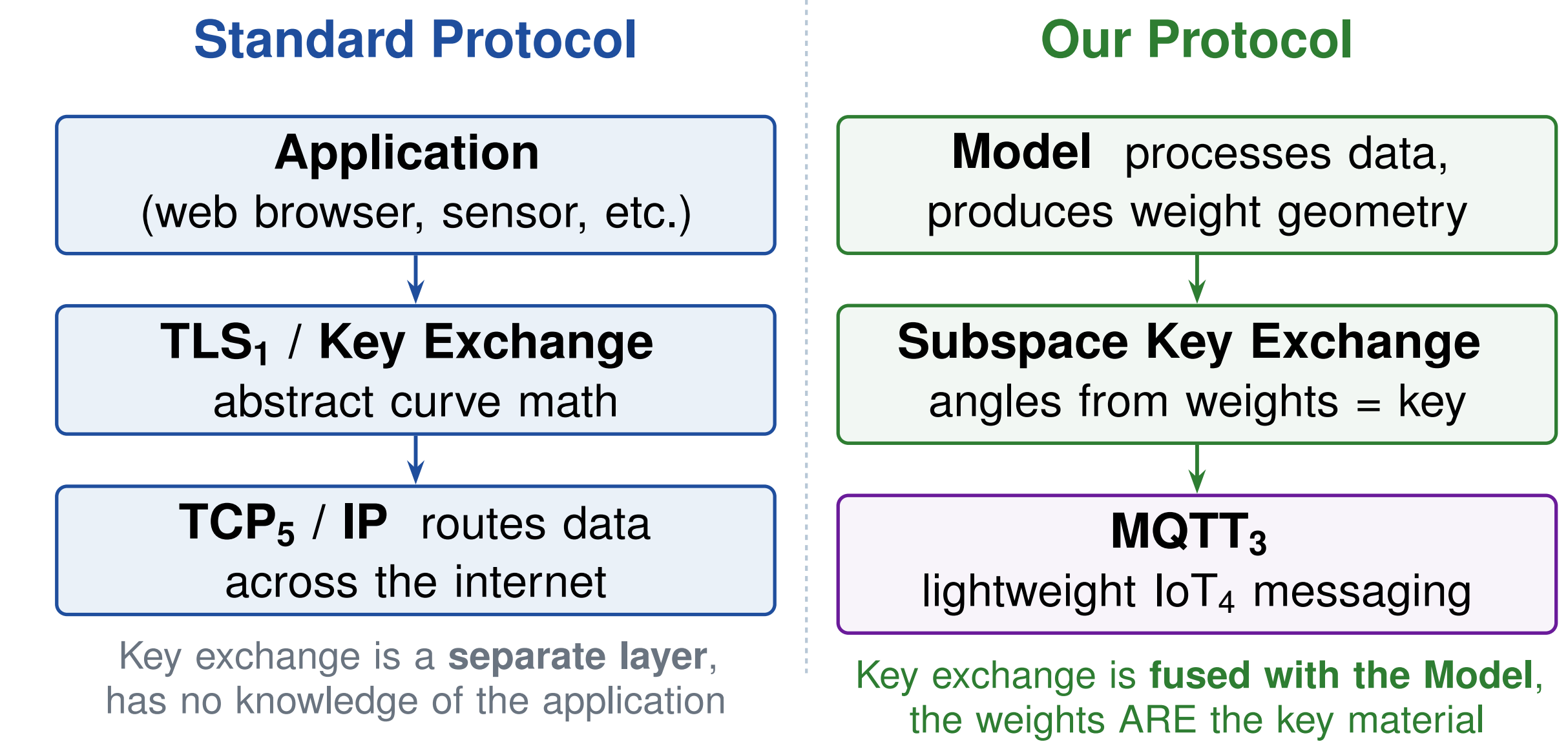


Figure 5. Left: in a standard protocol, the key exchange layer is generic math with no connection to the application. Right: in our protocol, the model's weight geometry is the key exchange, they are the same layer

Our Key Exchange vs. Standard

Feature	Standard (TLS ₁)	Our Method
Key source	Abstract curve math	model's weight geometry
Shared secret	A point on a curve	Angles between subspace slices
Key tied to the model?	✗ No, key is generic	✓ Yes, key IS the model
Forward secrecy	✓ New curve each session	✓ New random slice each session
Transport tested on	TCP ₅ (web traffic)	MQTT ₃ (IoT ₄ messaging)

Table 2. Standard key exchange is separate from the application. Ours is built into the model's geometry and tested over MQTT₃

Why did we choose MQTT₃?

MQTT₃ is a lightweight publish/subscribe messaging protocol designed for IoT₄ devices with limited power, memory, and bandwidth. Every message passes through a **central broker**. We take advantage of that architecture:

- The broker already connects every device, we use its topic system to route handshake messages without adding any new infrastructure
- Devices never need to know each other's IP addresses; the broker handles all routing
- Our key exchange messages look like ordinary MQTT₃ sensor data, an attacker watching the broker sees nothing unusual
- The broker only *forwards* encrypted payloads, it never sees the key or the plaintext, so a compromised broker cannot break the encryption